Γιατί χρειαζόμαστε βάσεις δεδομένων

- Με απλά αρχεία έχουμε διπλότυπα, ασυνέπειες και δύσκολη ταυτόχρονη πρόσβαση.
- Ένα ΣΔΒΔ προσφέρει ανεξαρτησία δεδομένων, ταυτόχρονη χρήση, ανάκαμψη μετά από βλάβη και έλεγχο πρόσβασης.
- Σχεσιακό μοντέλο: τα δεδομένα είναι σχέσεις (πίνακες), με πλειάδες (γραμμές) και γνωρίσματα (στήλες).
- Το σχήμα περιγράφει τη δομή, ενώ το στιγμιότυπο τα δεδομένα μιας δεδομένης στιγμής.

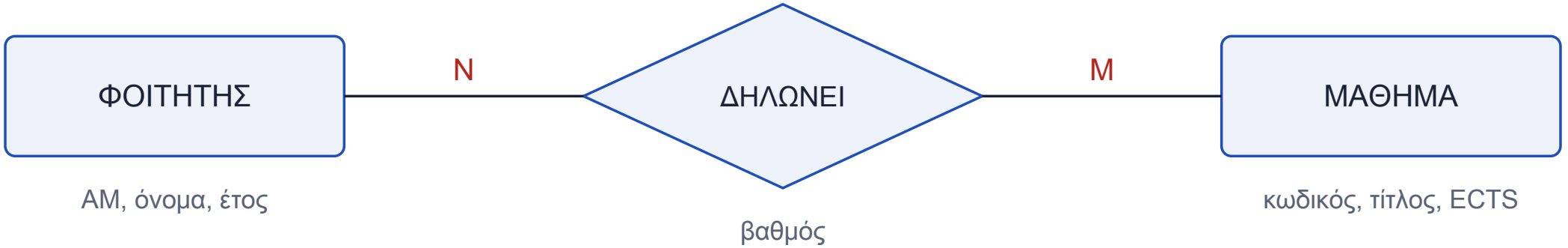
Κλειδιά

- Υποψήφιο κλειδί: ελάχιστο σύνολο γνωρισμάτων που προσδιορίζει μοναδικά μια πλειάδα. Πρωτεύον κλειδί: αυτό που επιλέγουμε, χωρίς τιμές NULL.
- Ξένο κλειδί: αναφέρεται στο πρωτεύον κλειδί άλλου πίνακα και εξασφαλίζει ακεραιότητα αναφορών.

```
CREATE TABLE student (
  am INT PRIMARY KEY, name VARCHAR(60) NOT NULL, year INT
);
CREATE TABLE enrollment (
  am INT REFERENCES student(am), code CHAR(6), grade DECIMAL(3,1),
  PRIMARY KEY (am, code)
);
```

Μοντέλο οντοτήτων-συσχετίσεων

- Οντότητες, γνωρίσματα και συσχετίσεις. Η πληθικότητα μπορεί να είναι 1:1, 1:N ή N:M.
- Μια συσχέτιση N:M γίνεται δικός της πίνακας, με πρωτεύον κλειδί τα δύο ξένα κλειδιά: εδώ ο enrollment.



SELECT: βασικά ερωτήματα

- Η σειρά εκτέλεσης είναι λογικά: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.
- Το NULL σημαίνει άγνωστη τιμή: γράφουμε IS NULL, γιατί το = NULL δεν γίνεται ποτέ αληθές.

```
SELECT name, year
FROM student
WHERE year = 2 AND name LIKE 'M%'
ORDER BY name;
```

Συνενώσεις (JOIN)

- INNER JOIN: μόνο οι γραμμές που ταιριάζουν και στους δύο πίνακες.
- LEFT JOIN: όλες οι γραμμές του αριστερού πίνακα, με NULL όπου δεν υπάρχει ταίριασμα.
- Έτσι βρίσκουμε φοιτητές χωρίς καμία δήλωση:

```
SELECT s.name
FROM student s LEFT JOIN enrollment e ON s.am = e.am
WHERE e.am IS NULL;
```

Ομαδοποίηση

- Οι συναθροιστικές συναρτήσεις: COUNT, SUM, AVG, MIN, MAX.
- Το WHERE φιλτράρει γραμμές πριν την ομαδοποίηση, ενώ το HAVING φιλτράρει ομάδες μετά.
- Το παράδειγμα δίνει τον μέσο όρο κάθε μαθήματος με τουλάχιστον 5 περασμένους φοιτητές:

```
SELECT code, AVG(grade) AS average
FROM enrollment
WHERE grade >= 5
GROUP BY code
HAVING COUNT(*) >= 5;
```

Γιατί κανονικοποίηση: ανωμαλίες

- Φανταστείτε έναν πίνακα δήλωση(ΑΜ, όνομα, κωδικός, τίτλος, βαθμός).

Ανωμαλία	Τι συμβαίνει στο παράδειγμα
Ενημέρωσης	Για να αλλάξει ο τίτλος ενός μαθήματος, αλλάζουμε πολλές γραμμές
Εισαγωγής	Δεν μπορούμε να καταχωρίσουμε μάθημα χωρίς να το έχει δηλώσει φοιτητής
Διαγραφής	Αν διαγραφεί ο τελευταίος φοιτητής ενός μαθήματος, χάνεται και ο τίτλος του

Συναρτησιακές εξαρτήσεις και κανονικές μορφές

- X → Y: όσες πλειάδες έχουν ίδιο X έχουν και ίδιο Y.

Μορφή	Απαίτηση
1NF	Κάθε γνώρισμα έχει ατομικές τιμές
2NF	1NF, και κανένα μη κλειδί δεν εξαρτάται από μέρος του κλειδιού
3NF	2NF, και καμία μεταβατική εξάρτηση μη κλειδιού από το κλειδί
BCNF	Για κάθε μη τετριμμένη X → Y, το X είναι υπερκλειδί

Παράδειγμα κανονικοποίησης

- Έχουμε δήλωση(ΑΜ, κωδικός, όνομα, τίτλος, βαθμός) με κλειδί (ΑΜ, κωδικός).
- Εξαρτήσεις: ΑΜ → όνομα, κωδικός → τίτλος, (ΑΜ, κωδικός) → βαθμός.
- Το όνομα και ο τίτλος εξαρτώνται από μέρος του κλειδιού, άρα ο πίνακας δεν είναι σε 2NF.
- Διασπάμε σε φοιτητής(ΑΜ, όνομα), μάθημα(κωδικός, τίτλος) και δήλωση(ΑΜ, κωδικός, βαθμός).
- Η διάσπαση πρέπει να είναι χωρίς απώλειες: με συνένωση παίρνουμε πίσω τον αρχικό πίνακα.

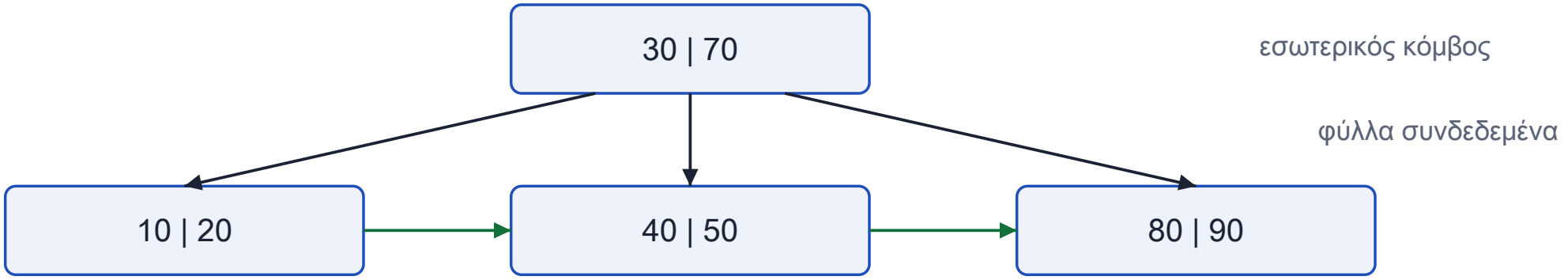
Συναλλαγές και ιδιότητες ACID

Ιδιότητα	Τι εγγυάται
Ατομικότητα (Atomicity)	Όλα ή τίποτα: σε αποτυχία γίνεται ROLLBACK
Συνέπεια (Consistency)	Η βάση περνά από έγκυρη κατάσταση σε έγκυρη κατάσταση
Απομόνωση (Isolation)	Οι ταυτόχρονες συναλλαγές δεν επηρεάζουν η μία την άλλη
Μονιμότητα (Durability)	Μετά το COMMIT, οι αλλαγές επιβιώνουν από βλάβη

```
BEGIN;
UPDATE account SET balance = balance - 100 WHERE id = 1;
UPDATE account SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

Ευρετήρια

- Χωρίς ευρετήριο, η αναζήτηση σαρώνει όλο τον πίνακα: O(n). Με B+ δέντρο: O(log n), και υποστηρίζει ερωτήματα εύρους, γιατί τα φύλλα είναι συνδεδεμένα.
- Το κόστος: επιπλέον χώρος και πιο αργές εισαγωγές και ενημερώσεις.



Ανακεφαλαίωση

- Το ξένο κλειδί εξασφαλίζει ακεραιότητα αναφορών. Μια συσχέτιση N:M γίνεται δικός της πίνακας.
- Το WHERE φιλτράρει γραμμές και το HAVING ομάδες. Για NULL γράφουμε IS NULL.
- Το LEFT JOIN κρατά όλες τις γραμμές του αριστερού πίνακα.
- 2NF: καμία εξάρτηση από μέρος του κλειδιού. 3NF: καμία μεταβατική εξάρτηση.
- ACID: ατομικότητα, συνέπεια, απομόνωση, μονιμότητα.
- Το ευρετήριο επιταχύνει τις αναζητήσεις, αλλά επιβαρύνει τις εγγραφές.